浙江大学



题　　目	IRON 手套
课程名称	数字系统实验
姓名与学号	梁凯文　3200104988
姓名与学号	宋智伟　3200103042
姓名与学号	王琪葳　3200102602
指导老师	叶德信　卓若凡
所在学院	信息与电子工程学院

2021 年 7 月 10 日

目录

一、设计理念..... 3

二、项目展示.....4

无线电灯带效果图.....4
时钟效果图.....4
电压自卫效果图.....4
无线电响指模块 5
LED 显示图模块.....5
机械杆舵机安装图.....5

三、功能介绍.....6

四、功能实现.....7

4.3 语音识别模块..... 15
4.4 语音播报模块..... 18
4.5 挥手开关模块..... 19
4.6 电压自卫模块..... 20
4.7 机械杆控制模块..... 20
4.8 时钟模块..... 21
4.9 LCD 屏幕显示模块..... 22
4.10 升降压和电平转换模块..... 24

五、心得体会.....25

（1）梁凯文：.....25
（2）宋智伟：.....26
（3）王琪葳：.....26

六、成员分工及贡献比.....28

梁凯文.....28
宋智伟.....28
王琪葳.....28

IRON 手套

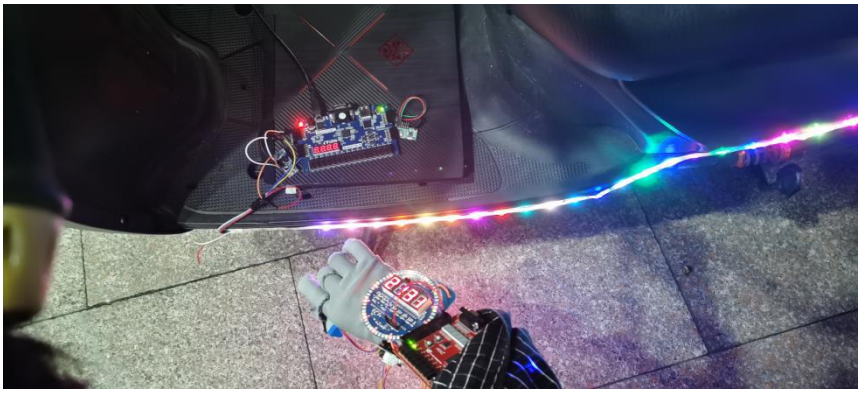
一、设计理念

随着计算机软硬件和互联网技术的高速发展，可穿戴式智能设备的形态开始多样化，逐渐在工业、医疗健康、军事、教育、娱乐等领域表现出广阔的应用潜力。而灵活的双手是造物主给予人类最好的礼物，各种巧妙的创造都能通过灵巧的十指完成，我们小组希望利用数字系统课程学习中的知识，让这种灵活的创造性更好的融合进科技之中，让一切皆有可能。

本产品的设计旨在提升学生在 ZJU 校园生活的便捷性及趣味性，让学生在日常生活中感受到智能生活的便利和温暖。相比于日常的手机，我们设计的手套设备在具备众多功能之外拥有不一样的特点：免持使用、即时提醒、与功能模块的自由组合搭载。因此，我们小组更加倾向将手套看作一个平台，各个模块以手套为载体，构成一款独特、便捷的多功能 ZJU 手套。

鉴于模块本身大小的限制、模块之间的连接限制、还有硬件和软件代码方面的局限性，我们小组无法使手套便携性得以充分的体现，但是，在现有的知识基础上我们小组竭尽全力，最大程度地完成手套各项功能的实现。这是我们对于所学知识赋予实践的初次尝试，也是我们在数字系统学习之路的坚实一步。

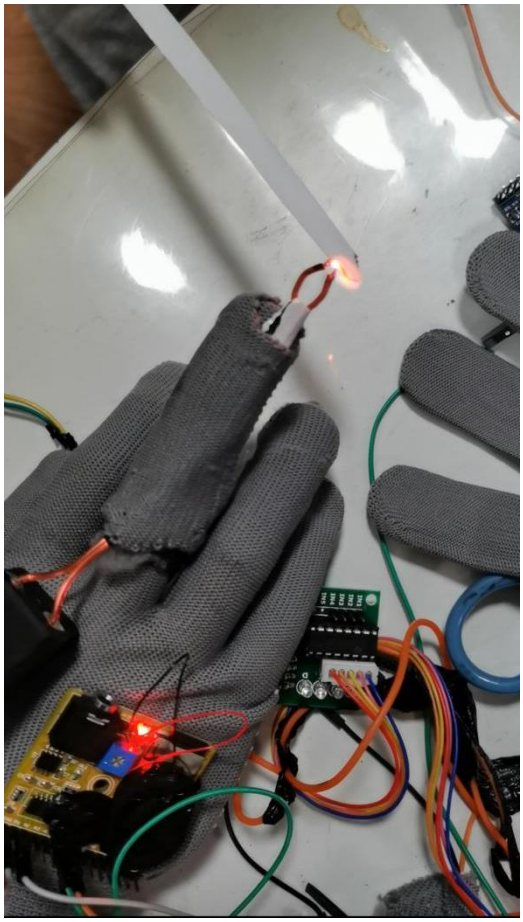
二、项目展示



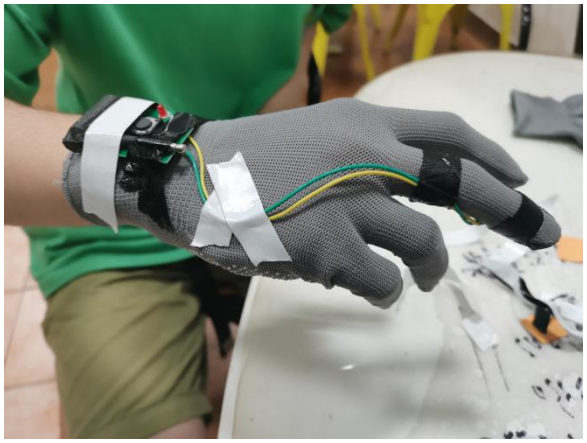
无线电灯带效果图



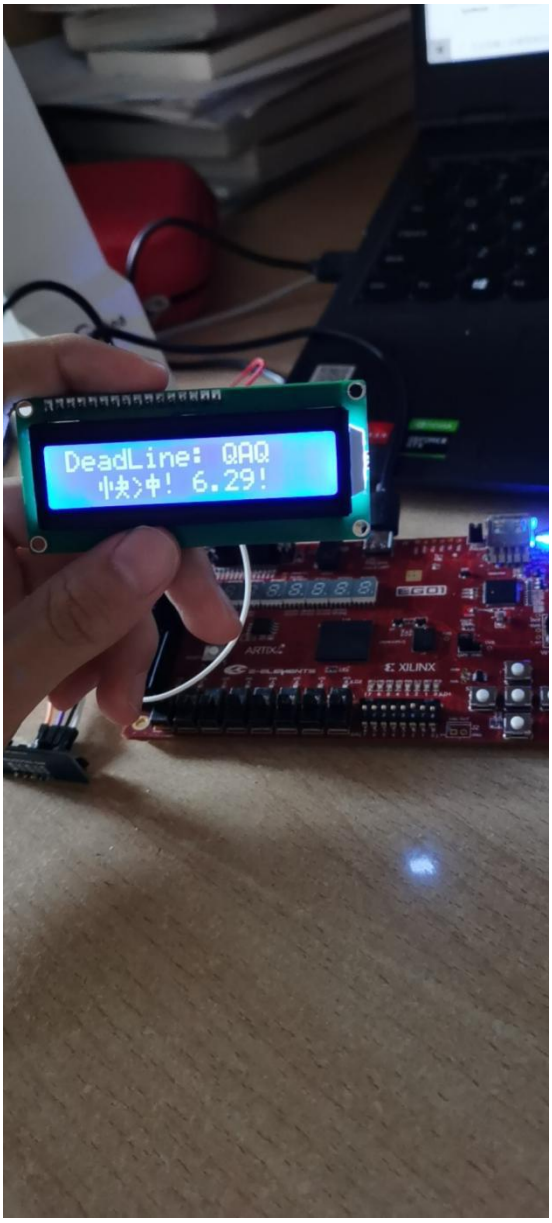
时钟效果图



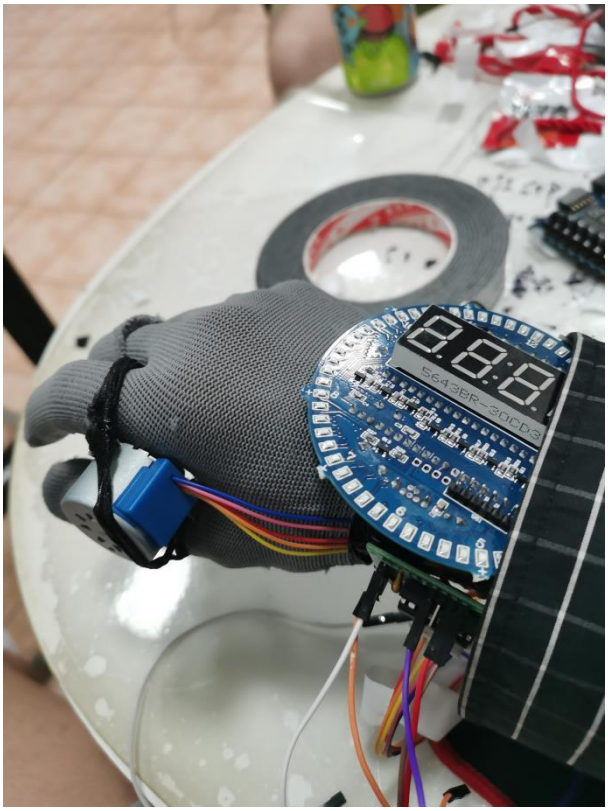
电压自卫效果图



无线电响指模块



LED 显示图模块



机械杆舵机安装图

展示部分模块，其余详见展示视频。

三、功能介绍

3.1 无线电模块

为了实现遥控打开自行车车灯带的效果，需要制作手套和电动车之间的通讯模块。由于应用场景是寻找自行车，不清楚自行车的位置且车库较为空旷，故不适合使用红外线遥控。使用无线电遥控，其遥控范围更加广阔，信号的接收也更加准确，即使场景较为空旷也能保证触发。保证每次在发射端发射信号，接收端都可以准确触发灯带光效。

3.2 led灯带显示模块

在车上需要安装显眼的灯带光效，使得当手套端发出寻找信号时，自行车端能够及时响应，在一众自行车中脱颖而出被主人找到。

3.3 语音识别模块

我们希望我们的产品与使用者有更好的交互体验。增添语音识别模块，增强产品使用的便捷性与趣味性。

3.4 语音播报模块

语音播报模块主要需要实现播放音频的功能，以及接收带其他模块的输入信号，将模块之间联系起来。

3.5 挥手开关模块

挥一挥衣袖，和自行车说再见。挥手开关需要实现感应挥手并将信号传输给FPGA控制板的功能，再由控制板驱动做出反应。

3.6 电压自卫模块

我们希望我们的手套如钢铁侠的盔甲一样，能够为使用者提供安全保障。手套上搭载电压自卫模块，按下按钮则释放高压，形成威慑击退贼人，保护自身安全。

3.7 机械杆控制模块

为了实现手套自动搅拌咖啡的功能，需要在手套上配备机械杆的控制模块，便利使用者的刷夜生活。

3.8 时钟模块

为了方便手套使用者对时间安排的把控，我们设计手套时钟模块，可详细准确地显示日期、时间，添加闹钟、计时器等功能，以上均可手动修改。同时，时钟搭载灯光加以美化，不同的灯光模式，让我们的手套炫酷多彩。

3.9 LCD 显示模块

LCD 显示的设计类似于可随身携带的备忘录，我们希望通过该功能，能够方便使用者记录生活中诸如 DDL、课表、考试信息等重要事宜，摆脱遗忘的烦恼。

3.10 升降压和电平转换模块

购入的模块大多是 5V 电压驱动，而板子的接口是 3.3V, 因此需要将电源电压升压到 5V 来给各个模块提供足够的电压。

模块之间，模块和板子之间也需要通信。为了更好地实现各功能模块的成功搭建，我们配置升压与电平转换模块，以期望实现无限搭载，无限可能的理念。

四、功能实现

4.1 无线电模块

手套上配备的无线电模块是用 1527 遥控芯片制成的遥控发射端。

在自行车上配备的是可以学习码配对的无线电接收模块。信号频率为 315HZ，通讯方式为调幅 AM。

无信号接收到时，其引脚全部为低电平。当接收端接收到对码后的无线电信号时，会在对应的引脚触发高电平，给手套上的 FPGA 板一个信号。



接收端引脚示意图

4.2 led灯带显示模块

4.2.1 模块原理说明

自行车端搭载的是WS2812B可编程灯带。灯带由5050型RGB LED灯组成，其中集成了非常紧凑的WS2812B LED驱动器IC。

WS2812B是一个集控制电路与发光电路于一体的智能外控LED光源。其外型与一个5050LED灯珠相同，每个元件即为一个像素点。像素点内部包含了智能数字接口数据锁存信号整形放大驱动电路，还包含有高精度的内部振荡器和可编程定电流控制部分，有效保证了像素点光的颜色高度一致。

其优点有如下几点：

- ✧ 内置信号整形电路，任何一个像素点收到信号后经过波形整形再输出，保证线路波形畸 变不会累加，颜色显示准确率高。
- ✧ 内置上电复位和掉电复位电路。
- ✧ 每个像素点的三基色颜色可实现256级亮度显示，完成16777216种颜色的全真色彩显示，扫描频率不低于400Hz/s，颜色选择非常多样。
- ✧ 串行级联接口，能通过一根信号线完成数据的接收与解码，对FPGA板接口数占用数少。
- ✧ 任意两点传传输距离在不超过5米时无需增加任何电路。布线方式灵活，非常适合在自行
- ✧ 车上装载。
- ✧ 光的颜色高度一致，性价比高。
- ✧ 灯带的显示效果可以通过编程实现，光效非常好看。



效果图

对于其数据的传输，数据协议采用单线归零码的通讯方式，像素点在上电复位以后，DIN端接受从控制器传输过来的数据，首先送过来的24bit数据被第一个像素点提取后，送到像素点内部的数据锁存器，剩余的数据经过内部整形处理电路整形放大后通过DO端口开始转发输出给下一个级联的像素点，每经过一个像素点的传输，信号减少24bit。

我们只需使用 FPGA 板上的一个引脚即可控制整个 LED 灯条。每个 LED 在每端有三个连接器接口，两个用于供电，一个用于数据。箭头表示数据流方向。前一个 LED 的数据输出焊盘连接到下一个 LED 的数据输入焊盘。我们可以将灯带切割成我们想要的任何尺寸，也可以使用电线增加 LED 灯距离。

至于供电，它们工作在 5V，每个红色、绿色和蓝色 LED 需要大约 20mA，或者在全亮度下每个 LED 总共 60mA。实验使用的 FPGA 板子是 3.3V 通信，因此在模块和板子相连接的时候需要升压模块和 3.3V-5V 电平转换模块。

4.2.2 Verilog 程序说明

（1）程序结构图



（2）顶层模块说明

```
module TOP(
    input   clk,
    input   rst_n,
    output  RZ_data
);
//------------------------------//
wire    tx_en;
wire    tx_done;
wire    [23:0]  RGB;
//------------------------------//
```

输入输出和内部定义的变量

```
//------------------------------//

RGB_Control2        RGB_Control_inst0(
    .clk        (clk),
    .rst_n      (rst_n),
    .tx_done    (tx_done),
    .tx_en      (tx_en),
    .RGB        (RGB)
);
//------------------------------//
RZ_Code         RZ_Code_inst1(
    .clk        (clk),
    .rst_n      (rst_n),
    .RGB        (RGB),
    .tx_en      (tx_en),
    .tx_done    (tx_done),
    .RZ_data    (RZ_data)
);
endmodule
```

函数接口定义

（3）RGB_Control 模块说明

本模块是对整条灯带进行操作，实现对灯与灯之间的转换的控制。

```
module RGB_Control2(
    input   clk,
    input   rst_n,
    input   tx_done,        //一帧24bit)数据结束标志
    output  tx_en,          //发送数据使能
    output  reg [23:0]  RGB
);

//-------------接口信号------------//
reg    [31:0]  cnt;
reg    [23:0]  RGB_reg [4:0];  //存RGB数据的数据
reg    [7:0]   k;
reg    tx_en_r;
//---------------------------------//
reg    tx_done_r0;
```

模块的输入输出和内部定义的寄存器变量

输入信号：

clk 为板子时钟信号

rst_n 为复位信号

tx_done 是从 RZ_Code 传来的信号，当 RZ_Code 中将一帧 24bit 数据发送到灯带端 完成，tx_done 传入为 1，否则为 0

输出信号：

tx_en 为向 RZ_Code 中传输的是否发送 24bit 数据的使能信号

24 位的 RGB 信号是向 RZ_code 中发送的下一个灯的 24bit 显示信息

每一个灯的 24bit 数据储存：

```
always @(posedge clk or negedge rst_n) begin
    if(rst_n)   begin
        k <= 0;
        RGB <= 0;
        RGB_reg [0] <= 24'b11111111_00000000_11111111;
        RGB_reg [1] <= 24'b00000000_11111111_00000000;
        RGB_reg [2] <= 24'b10101010_01010101_10101010;
        RGB_reg [3] <= 24'b10100101_01000011_11010101;
        RGB_reg [4] <= 24'b10100101_01000011_11010101;
        RGB_reg [5] <= 24'b11111111_00000000_11111111;
```

24bit 数据储存

cnt 计数控制：

使用 cnt 来控制传输数据速度。如果没有 cnt 计数控制灯与灯之间传输的间隔时间， 灯带显示的灯光就会过快，不能完成对每个灯带的显示控制。

```
//-------------计数-------------//
always @(posedge clk) begin
    if((rst_n) || (tx_en_r))    begin
        cnt <= 0;
    end
    else if(cnt == 32'd14999)   begin   //RESET时间(300us*50M=15000)
        cnt <= cnt;
    end
    else    begin
        cnt <= cnt + 1;
    end
end
//--------------------------------//
```

cnt 计数器代码图

led 灯操作对象转换控制：

```
else if(tx_en_r)    begin
    case (k)
                8'd0,8'd1,8'd2,8'd3,8'd4,8'd5,8'd6,8'd7,8'd8,8'd9,8'd10,
                8'd11,8'd12,8'd13,8'd14,8'd15,8'd16,8'd17,8'd18,8'd19,8'd20,
                8'd21,8'd22,8'd23,8'd24,8'd25,8'd26,8'd27,8'd28,8'd29,8'd30,
                8'd41,8'd42,8'd43,8'd44,8'd45,8'd46,8'd47,8'd48,8'd49,8'd50,
                8'd51,8'd52,8'd53,8'd54,8'd55,8'd56,8'd57,8'd58,8'd59,8'd60,
                8'd61,8'd62,8'd63,8'd64,8'd65,8'd66,8'd67,8'd68,8'd69,8'd70,
                8'd71,8'd72,8'd73,8'd74,8'd75,8'd76,8'd77,8'd78,8'd79,8'd80,
                8'd81,8'd82,8'd83,8'd84,8'd85,8'd86,8'd87,8'd88
            :
            if(tx_done_r0)  begin
                RGB <= RGB_reg[k];
                k <= k + 1;
            end
        8'd89:
            if(tx_done) begin
                RGB <= RGB_reg[0];
                k <= 0;
            end
        default: k <= 0;
    endcase
end
else ;
end
```

状态转换

使能控制：

使用 tx-en 来完成对是否发送下一个 led 灯 24bit 数据的使能控制。

灯带长度为 90 个 led 灯，使用 k 计数来判断一组 90 个灯的数据是否发送完成，k 的初始值为 0，每当一个灯的 24bit 数据发送给灯带端完成，k+1 。当 k=89 时，说明 90 个灯全部发送完成，于是重新开始下一轮发送。

```
//--------------------------------//
always @(posedge clk or negedge rst_n) begin
    if(rst_n)   begin
        tx_en_r <= 0;
    end
    else if((k == 8'd89) && (tx_done))  begin
        tx_en_r <= 0;           //一组数据发送结束，RESTE
    end
    else if((cnt == 32'd14999) && tx_done)  begin
        tx_en_r <= 1;           //重新开始发送
        end
    else
    begin
        tx_en_r <= tx_en_r;
    end
end

assign  tx_en = tx_en_r;
//--------------------------------//
```

使能控制部分代码图

（4）RGB_Code 模块说明

本模块对目前操作的 led 灯对象进行数据传输。首先将 01 数据转化为单极性归零码，再单比特传输 24bit 数据到灯带端实现一个灯的点亮。

```
//  输入数据为RGB数据(24bit)
//  输出数据为单极性归零码(Return Zero Code)

//本模块是对每一个灯进行操作
module RZ_Code(
    input   clk,
    input   rst_n,
    input   [23:0]  RGB,    //按照GRB的顺序排列
    input   tx_en,          //发送数据使能
    output  tx_done,        //一帧(24bit)数据结束标志
    output  RZ_data
);

//-----------------接口信号----------------//
reg     [31:0]  cnt = 0;        //计数一个码元周期
reg     [4:0]   i = 0;          //计数24bit
reg     symbol = 0;             //1bit数据结束标志
reg     RGB_RZ = 0;             //需要发送的RGB数据
reg     data_out = 0;           //转换后的单极性归零码
```

模块的输入输出和内部定义的寄存器变量

输入信号：

clk 为板子时钟信号

rst_n 为复位信号

tx_en 为 RGB_Control 中传来的是否发送 24bit 数据的使能信号

输出信号：

24 位的 RGB 信号是向 RZ_code 中发送的下一个灯的 24bit 显示信息

tx_done 是传向 RGB_Control 的信号，当 RZ_Code 中将一帧 24bit 数据发送到灯带端完成，tx_done 传出为 1，否则为 0

码元周期计数器：

一个码元周期 1，25us * 100M 的时钟，得出一个码元周期为 125ns 。

```
//-----------------计数器-----------------//
always @(posedge clk or negedge rst_n) begin
    if(rst_n)   begin
        cnt <= 0;
    end
    else if(cnt == 32'd125) begin       //计数一个码元周期(125=1.25us * 100M)
        cnt <= 32'd0;
    end
    else    begin
        cnt <= cnt + 1'b1;
    end
end
```

码元周期计数器

单极性归零码：

```
//-------------单极性归零码---------------//
always @(posedge clk ) begin
    if((rst_n) || (!tx_en)) begin
        data_out <= 0;
    end
    else if((RGB_RZ == 0) && (tx_en == 1))  begin
        if(cnt <= 32'd30)   begin       //零码高电平，0.3us*100M=30
            data_out <= 1;
        end
        else    begin
            data_out <= 0;
        end
    end
    else if((RGB_RZ == 1) && (tx_en == 1))  begin
        if(cnt <= 32'd90)   begin       //一码高电平，0.9us*100M=90
            data_out <= 1;
        end
        else    begin
            data_out <= 0;
        end
    end
    else    begin
        data_out <= data_out;
    end
end
```

单极性归零码

码元周期结束标志 **symbol** ：

```
always @(posedge clk or negedge rst_n) begin
    if(rst_n)   begin
        symbol <= 0;
    end
    else if(cnt == 32'd125) begin       //码元周期结束使symbol = 1
        symbol <= 1;
    end
    else    begin
        symbol <= 0;                    //下一个码元周期开始时使symbol为0
    end
end
//一个码元周期结束则symbol赋值为1，下一个码元周期开始时使symbol为0
```

码元周期结束标志

码元发送状态转换：

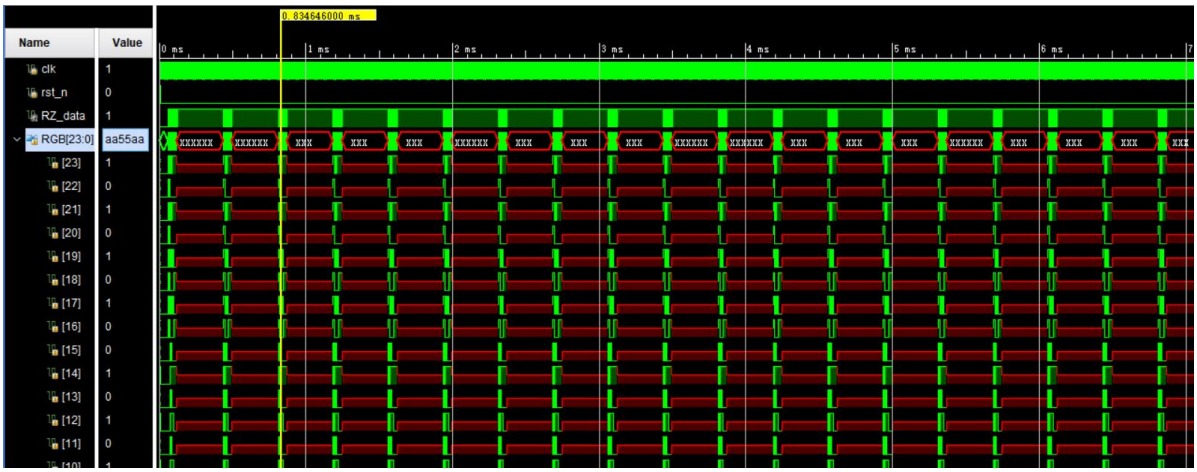
```
//--------------循环发送一帧--------------//
always @(posedge clk or negedge rst_n) begin
    if(rst_n)   begin
        i <= 0;
        RGB_RZ <= 0;
    end
    else    begin
        case (i)
            5'd0,5'd1,5'd2,5'd3,5'd4,5'd5,5'd6,5'd7,5'd8,5'd9,5'd10,5'd11,5'd12,5'd13,5'd14,5'd15,5'd16,5'd17,5'd18,5'd1
                if(symbol)  begin       //当一个码元周期结束时，为这个灯发送新的码元
                    i <= i + 1;
                    RGB_RZ <= RGB[23 - i];      //从高位到低位依次进行赋值
                end
                else    begin
                    i <= i;
                    RGB_RZ <= RGB_RZ;       //symbol为0说明还在这一码元周期内，给灯的码元不变
                end
            default:    i <= 0; //到第24位时先把i归零，再重新计数下一码元

        endcase
    end
end
//--------------------------------------//
```

发送码元对象转换

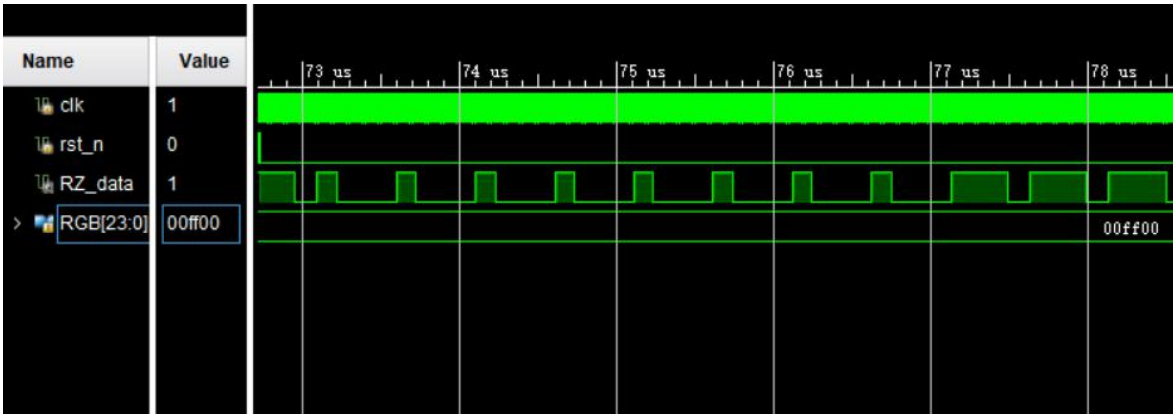
4.2.3 Verilog 程序仿真

在仿真图中可以很明显看到 cnt 的作用，红色部分就是计数等待时间。



仿真图

下图可以清楚地看到单极性归零码的方波图。



仿真图

4.3 语音识别模块

内容作业中用到的语言识别模块是目前市场上使用较为广泛的LD3320语言识别模块，LD3320（LDV7）语音模块可以工作在以下三种模式：

- 普通模式：直接说话，模块直接识别；
- 按键模式：按键触发开始ASR进程；
- 口令模式：需要一级唤醒词（口令）；

在作业中为了实现人机交互，主要使用的口令模式，具体功能是：在对语音识别模块叫出预先设置好的指令之后，可以唤醒语音识别模块，每次的唤醒指令都一致，再次说出预先设置的二级指令，比如说开灯，关灯之类的，此时可以按要求对代码进行修改，从而输出所需的高低电平。

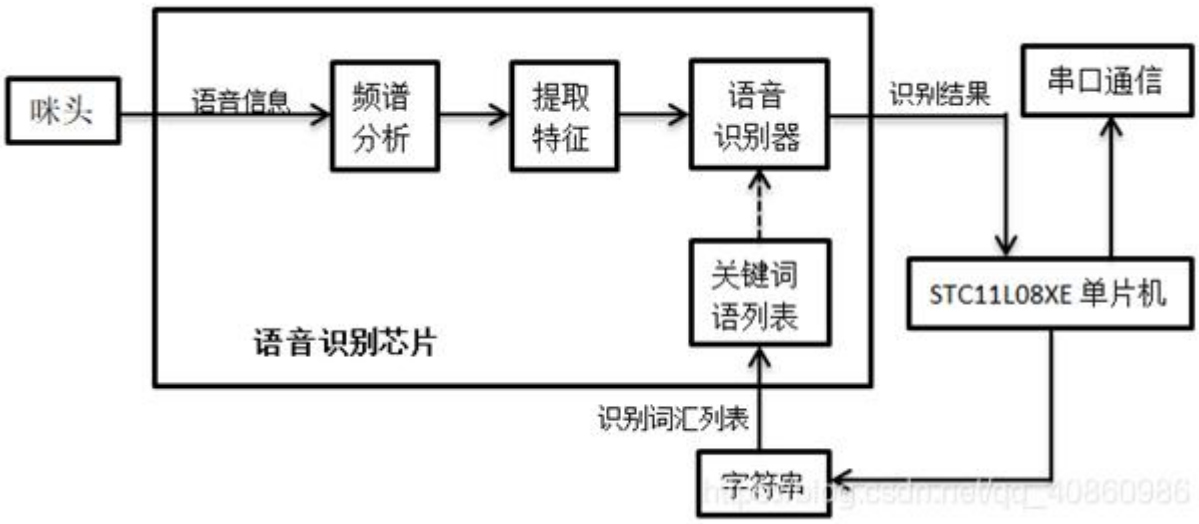
"jia wei si",\ "qi xiao ge",\ "guan deng",\ "da kai shou tao shang de deng",
"di di ai ao",
"xue bu dong le",
"shuo tian jin hua",\ "zao shang hao",

这段代码可以实现对一级指令和二级指令的修改，通过对字符做大长度的修改，可以设置字符长度以内的指令，但相对来说字符长度越长，灵敏度越低。我说：打开手套上的灯时语音模块就置 P1.3 为高电平。

P1.3 和 stm32 的一个引脚相连。当 STM32 检测到与 P1.3 相连引脚变为高电平，则控制 LED 灯打开。

对于词汇的识别问题，模块中可以对垃圾词汇进行编辑，当语音识别识别到垃圾词汇时会对垃圾词汇忽略。

模块工作图如下：



串口通信

（语音模块串口发送的内容）

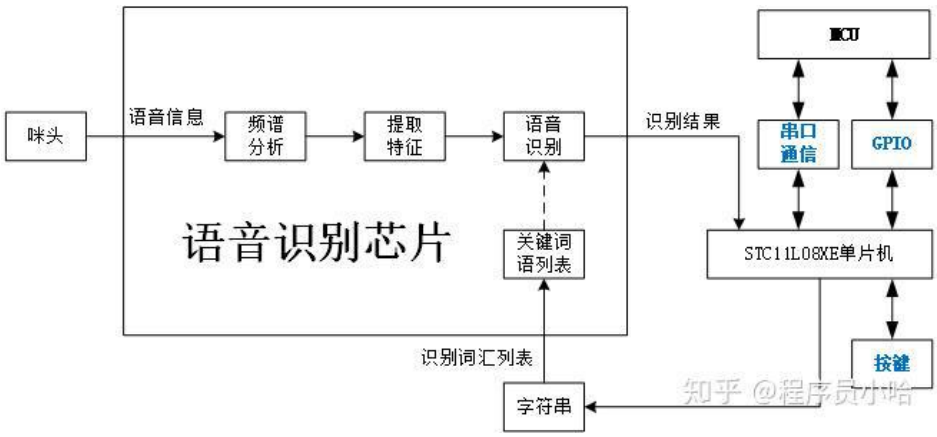
switch(dat) /*对结果执行相关操作,客户可删除 Printcom 串口输出语句替换为其他需要控制的代码*/

```
{
case CODE_DMCS: /*命令“ 亮一点” */
PrintCom("亮一点\r\n"); /*text.....*/
break;
case CODE_KFBYZ: /*命令“ 变暗” */
PrintCom("变暗\r\n"); /*text.....*/
break;
case CODE_KD: /*命令“ 开灯” */
PrintCom("开灯\r\n"); /*text.....*/
break;
case CODE_GD: /*命令“ 关灯” */
PrintCom("关灯\r\n"); /*text.....*/
break;
case CODE_BJ: /*命令“ 亮度30” */
PrintCom("亮度30\r\n"); /*text.....*/
break;
case CODE_SH: /*命令“ 亮度60” */
PrintCom("亮度60\r\n"); /*text.....*/
break;
case CODE_GZ: /*命令“ 亮度100” */
PrintCom("亮度100\r\n"); /*text.....*/
break;
default:PrintCom("请重新识别发口令\r\n"); /*text.....*/break;
}
}
else
{
PrintCom("请说出一级口令\r\n"); /*text.....*/
}
```

这部分代码是例程，并非实际使用，具体功能是能够在电脑上的串口上显示出指令是否识别成功，比如说我说开灯，识别成功后就有开灯识别成功的串口输出。

同时，语音识别是基于当下市场实用广泛的 LDS3320 语音识别模块，通过对代码的编辑和烧录，更改语音识别里面对应词条，可识别五十段口令识别，根据对 IO 口的编辑，将词条与 IO 口对应，实现在模块识别了词条之后按照需要输出高电平或者低电平，也可以外接播报进行识别播放。

工作原理如下：



即，你说一句话，然后模块用拼音去和你的发音做比较，在关键词中找出哪些预设值的词语与发音的接近程度大于多少的（假定 50%），然后再从一堆大于（50% ）里面输出一个最接近的。 这里就会造成一个问题，容易造成误识别太多。如果你只有一个关键词：时间 然后你说了一句：‘时刻’，他识别到‘时’这个发音，所以有 50% 以上的相似，但是候选词中没有‘时刻’这个比他更接近的关键词，所以他就会输出‘时间’的结果。这样子他就是错误了。 如果你设置一些拟声词，有可能随随便便就触发了。解决方案就是通过编辑“垃圾关键词汇”，从而使得避免垃圾词汇的筛选。在作业中实现的功能是将语音识别与语音播报模块拼接起来，将语音识别的输出作为语音播报的输入，实现人机互动，效果类似于天猫精灵这样，可以对人说出的话做出相应的反映。

4.4 语音播报模块

我们使用的语音播报模块主要是通过编码设置，将对应音频放入语音播报模块之后，对语音播报模块的 IO 口进行编辑，符合接口协议，对文件名进行修改，使用 mp3 格式的音频，得到效果是当接收到一个高电平是模块触发，播放音频里的歌曲。

工作的流程如下：

① 支持采样率(KHz):8/11.025/12/16/22.05/24/32/44.1/48
② 24 位 DAC 输出，动态范围支持 90dB，信噪比支持 85dB
③ 完全支持 FAT16、FAT32 文件系统，最大支持 32G 的 TF 卡，支持 32G 的 U 盘
④ 多种控制模式可选。4 个 IO 控制模式、串口模式；
⑤ 广播语插播功能，可以暂停正在播放的背景音乐。广告播放完毕回到背景音继续播放
⑥ 工作电压 8V-24V;
⑦ 音量可调，通过电位器调节；

工作图片如下：

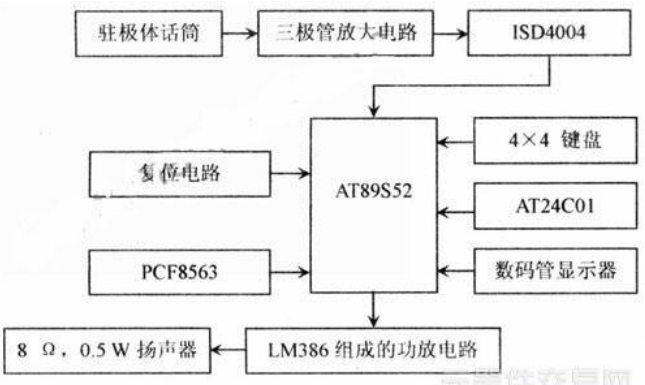
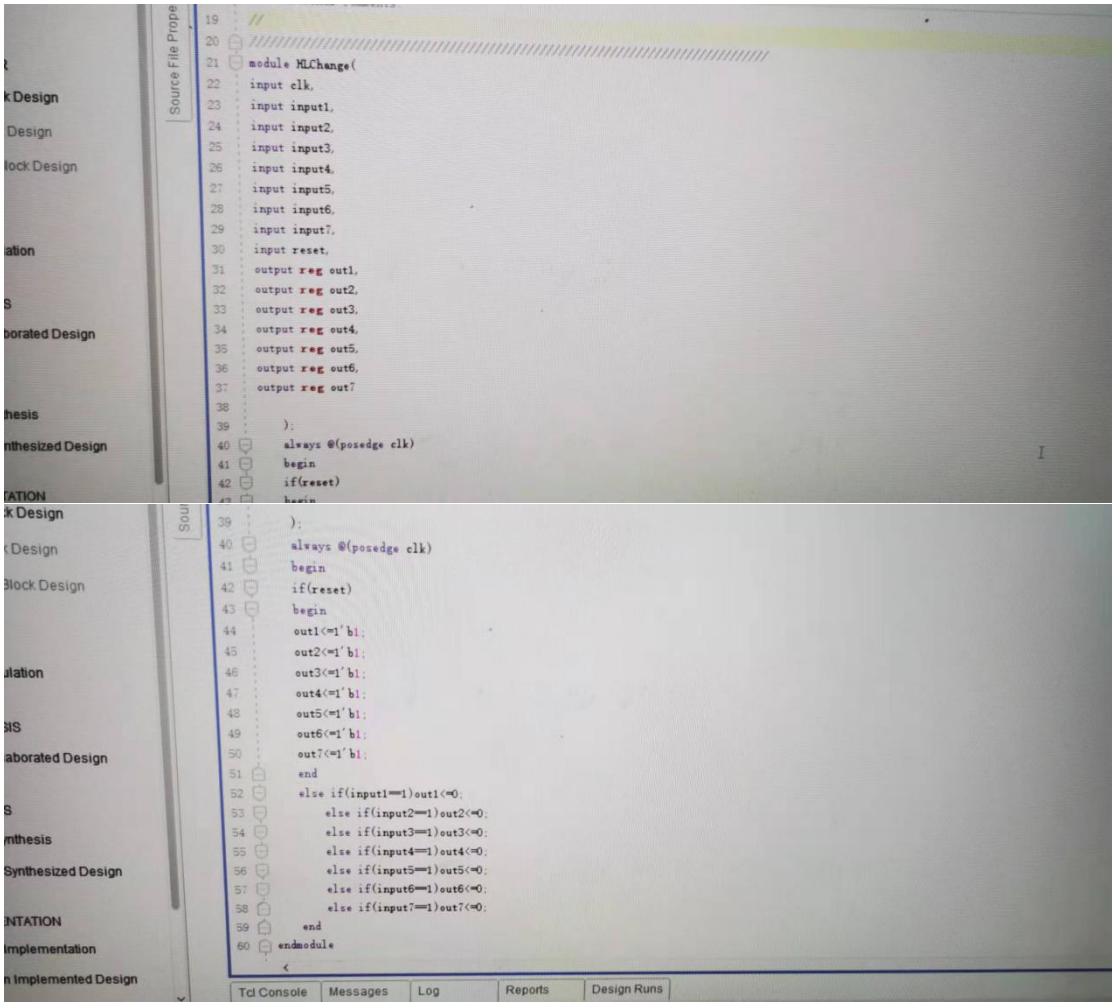


图 1 系统组成框图

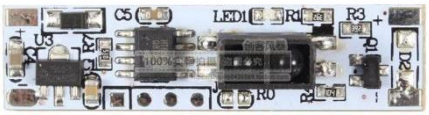


```
//
//////////////////////////////////////////////////////////////////////////////////
module HLChange(
input clk,
input input1,
input input2,
input input3,
input input4,
input input5,
input input6,
input input7,
input reset,
output reg out1,
output reg out2,
output reg out3,
output reg out4,
output reg out5,
output reg out6,
output reg out7

    );
    always @(posedge clk)
    begin
    if(reset)
    begin
    out1<=1'b1;
    out2<=1'b1;
    out3<=1'b1;
    out4<=1'b1;
    out5<=1'b1;
    out6<=1'b1;
    out7<=1'b1;
    end
    else if(input1==1)out1<=0;
        else if(input2==1)out2<=0;
        else if(input3==1)out3<=0;
        else if(input4==1)out4<=0;
        else if(input5==1)out5<=0;
        else if(input6==1)out6<=0;
        else if(input7==1)out7<=0;
    end
endmodule
```

这两张图片是前面提到我的语音播报模块输出不符合小组需求，询问老师后做出的改变，想从对 FPGA 的控制上入手，调剂输入与输出，计划是将 IO 口起始设置为高电平，有输入后转为低电平，不过在综合的过程中遇到了困难，电平一直保持高电平或者低电平，由于时间赶，又想了其他方法来替代。

4.5 挥手开关模块



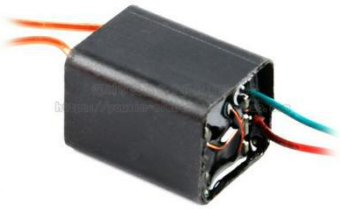
模块实物图

左接电源，右接输出。

利用红外感应，当挥手划过感应模块时，触发输出高电平。

4.6 电压自卫模块

采用特斯拉线圈原理制成的脉冲直流高压发生器 FDH-002，
输入 3.7V-6V 电压可以在两条线间输出形成 20KV 左右的高压。
经过测试可以直接点燃纸巾。



高压发生器

4.7 机械杆控制模块

使用五线四相直流舵机控制搅拌杆的旋转。
Dir 控制旋转方向顺时针还是逆时针。
设置 8 个状态和几个状态间的状态转换。使用状态机的方式控制舵机的旋转。
计数器的数值大小不能太小，否则舵机无法提供这么高的旋转速度，导致无法转动。
计数器和 StepEnable 的原理与灯带 cnt 原理相同，不再赘述。

```
module StepMotorPorts (StepDrive, clk, Dir, StepEnable, rst);

    input clk;
    input Dir;
    input StepEnable;
    input rst;

    output[3:0] StepDrive;
    reg[3:0] StepDrive;
    reg[2:0] state;
    reg[31:0] StepCounter = 32'b0;
    parameter[31:0] StepLockOut = 32'd80000;            //500HZ
    reg InternalStepEnable;

   always @(posedge clk or negedge rst)
        begin
       if ( rst)
       begin
           StepDrive      <= 4'b0;
           state    <= 3'b0;
           StepCounter <= 32'b0;
          end
```

```
    else
      begin
        if (StepEnable == 1'b1)    InternalStepEnable <= 1'b1 ;

         StepCounter <= StepCounter + 31'b1 ;
         if (StepCounter >= StepLockOut)
         begin
             StepCounter <= 32'b0 ;

             if (InternalStepEnable == 1'b1)
             begin
                 InternalStepEnable <= StepEnable ;
                 if (Dir == 1'b1)           state <= state + 3'b001 ;
                 else if (Dir == 1'b0)       state <= state - 3'b001 ;
                 case (state)
                     3'b000 :    StepDrive <= 4'b0001 ;
                     3'b001 :    StepDrive <= 4'b0011 ;
                     3'b010 :    StepDrive <= 4'b0010 ;
                     3'b011 :    StepDrive <= 4'b0110 ;
                     3'b100 :    StepDrive <= 4'b0100 ;
                     3'b101 :    StepDrive <= 4'b1100 ;
                     3'b110 :    StepDrive <= 4'b1000 ;
                     3'b111 :    StepDrive <= 4'b1001 ;
                 endcase
             end
         end
      end
  end

endmodule
```

4.8 时钟模块

基于 FPGA 的时钟模块的设计的各个功能在实现层面是并行的，通过 FPGA 板的时钟信号作为敏感变量，同时利用 flag 的设置来进行总体的控制。为实现时钟模块功能的正常实现，需要设计分频模块、自动计时模块、手动调整模块、闹钟设置模块，并设计相应的可视化数码管模块。

```
    begin
        if(op1<9)
            op1<=op1+1'b1;//秒钟最低位小于9，就加一，否则最低位变为0
        else
            begin
                if(op2<5)
                    op2<=op2+1'b1; //秒钟最低位为9时，十位是否为5，若小于5，十位加一，如：09-10，29-30
                                   //否则即为 59，->00.后两位为00分位置加一。
                else
                    begin
                        if(op3<9)
                            op3<=op3+1'b1;//由于分和秒都是60进制，因此方法同上。
                        else
                            begin
                                if(op4<5)
                                    op4<=op4+1'b1;
                                else
                                    op4<='d0;
                                op3<='d0;
                            end
                            op2<='d0;
                        end
                    op1<='d0;
                end
            end
    end
```

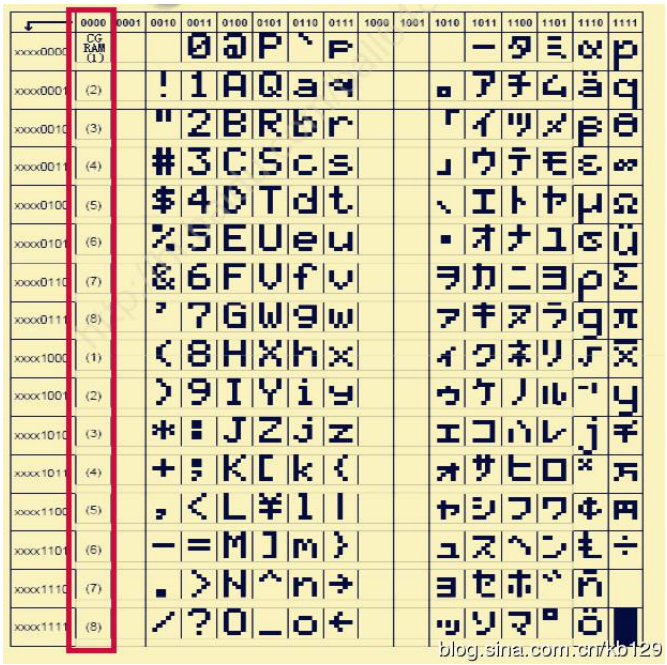
考虑到便携、醒目性，还有其他手套其他功能的同时实现，我们小组将部分时钟单独取出，使用设计好的由单片机驱动的整体闹钟，传入、设定相应的数据，调节闹钟的灯光。为了深入了解与学习，将计时器功能单独由 FPGA 数码管实现。该功能需要利用到时钟信号的分频，以分频出的时钟信号作为驱动，设计相应的状态机，实现不同位数上的逢 10 进 1，在数码管上显示，总体的实现相对简单。

4.9 LCD 屏幕显示模块

我们小组采用的是 LCD1602 模块，FPGA 驱动。作为一个独立的硬件，LCD1602 内置了 DDRAM、CGROM、CGRAM，其中要让 lcd 显示就需要将 DDRAM 的相应地址写入数据。LCD1602 可以显示上下两行，每一行显示 16 符号，一共显示 32 个符号，每个显示对应于 DDRAM 一个地址，若要实现显示，需要查询相对应 DDRAM 的地址，并写入相应数据。

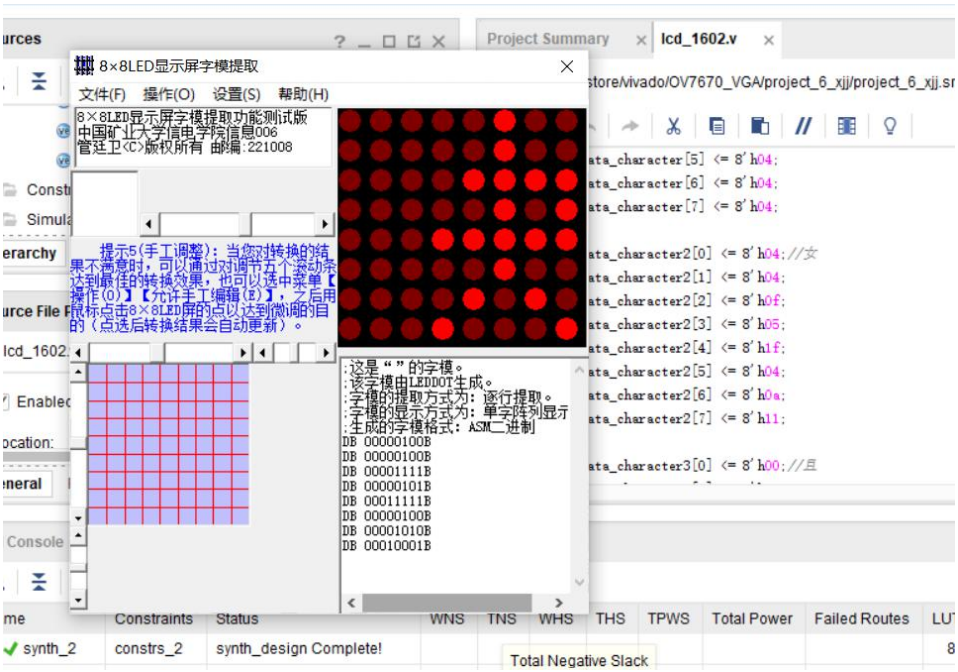
	显示位置	1	2	3	4	5	6	7	……	40
DDRAM 地 址	第一行	00H	01H	02H	03H	04H	05H	06H	……	27H
	第二行	40H	41H	42H	43H	44H	45H	46H	……	67H

这里的数据涉及 CGROM 和 CGRAM 的应用。因为 LCD1602 内置了 192 个常用字符的字模，存于 CGROM（character generate ROM）中，同时还有 8 个允许用户自定义字符（也就是可以显示八个中文字）的 RAM，也就是 CGRAM 可以利用。



如上图，若要显示“A”，则对应的数据为 01000001，转换为 16 进制“41H”输入对应的 DDRAM 地址即可，可以说是查找显示很是方便。

上图红框里面表示为 CGRAM，是可以使用者自行定义的字符，解决想显示汉字而在图中找不到汉字的问题。



先用字模软件，将对应汉字的变为二进制数。如上图便是我实现“快”字的右半部分二进制的转换。再利用 LCD1602 的 CGRAM 的写入指令储存到对应可调用的地址中。

这里需要说明的是，LCD1602 的显示有很多种选择，一共有 11 条指令，可以根据不同设置达到不同的显示，本质上自我认为是操作码+操作数的基本原理。



7.设定 CGRAM 地址指令

指令功能	指令编码										执行时间 /us
	RS	R/W	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
设定CGRAM地址	0	0	0	1	CGRAM的地址(6位)						40

功能：设定下一个要存入数据的 **CGRAM** 的地址。

DB5DB4DB3 为字符号，也就是你将来要显示该字符时要用到的字符地址。**(000~111)**（能定义八个字符）

DB2DB1DB0 为行号。（**000~111**）（八行）

blog.sina.com.cn/kb129

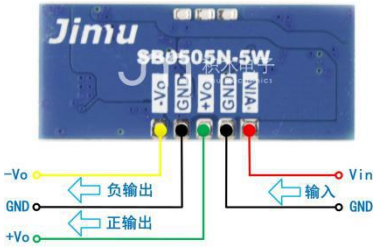
在实验过程中，还需要注意到时钟的问题，LCD1602 需求时序较低，因此我将 100MHz 的时钟信号分频为 500Hz，得以成功实现。如下图便是 CGRAM 利用状态机的写入部分代码。

```
write_cgram:
    begin
        case (num)
            0:begin
              data  <= 8'h40;       //
              num   <= num +1;
              state <= write_cgram;
            end
            1:begin
                if(write_count == 8)
                  begin
                    data  <= 8'h48;   //
                    RS    <= 1'b0;
                    num   <= num +1;
                    state <= write_cgram;
                    write_count<=0;
                  end
                else
                  begin
                    data <=data_character[write_count];
                           RS   <=1'b1;
                    write_count <= write_count+1'b1;
                    state  <= write_cgram;
                  end
            end
             2:begin
                     if(write_count==8)
```

4.10 升降压和电平转换模块

升降压模块使用下图所示直流升降压装置，可将 3.2V-14V 直流电压统一输出为±5V。

用于模块和板子间电平信号通讯的电平转换模块，选择了这款 4 路 3.3V-5V 双向电平转换模块。



升降压装置



双向电平转换模块

五、心得体会

（1）梁凯文：

最初开题的时候，我们要实现的功能太多太杂，老师建议我们剔除掉一些用处不大的小功能。

于是我们将涉及到比较复杂的机械控制的部分删除，在机械控制方面只保留了自动搅拌咖啡的机械杆。

后续在制作过程中，我们也发现在短短一两个月内，做出所有第一次开题时提到的所有功能不太现实。各种大问题小问题不断出现，从软件仿真跑不出来，到跑出来后实物上无法实现，甚至买来的硬件也有问题。

我不禁有些自责，作为组长，当初敲定选择这个方向，是不是我的错误？

不过，正如我们在开题时做出的承诺：“点子不一定都能实现，但我们会尽力去完成。” 即使屡屡碰壁，大家依然都硬着头皮去做，一遍又遍研究商家给的资料，一次又次尝试在硬件实现。断断续续做了半个多月后，在助教的帮助下，我发现仿真跑不出来居然是仿真时长不够的问题！

仿真成功之后，实现并不是水到渠成了。使用了电平转换模块后，电压依然始终不足以驱动灯带。在助教的提醒下，我添加了升压模块去驱动灯带。又一次忐忑地按下无线电遥控按钮后… 突然，绚丽的光效在昏暗的书桌前展开，灯带被点亮了！

好像按下了好运按钮一般，这之后好消息开始不断出现，智伟的语音播放也实现了，琪葳的 lcd 显示屏终于也被点亮了…

我们越来越有信心，之后问题虽然依然一个个出现，我们却有了解决它们的勇气和决心。我去反复思考怎么将大家各个模块之间连接起来，去解决语音接地问题，去解决升压和电平转换问题，去搞外接电源，去改线布线制作手套，去为舵机寻找合适的驱动模块……

我们去着手解决百思不得其解的难题，去应对一次次突发情况。一个多月后，我们回过头来看，当初的功能竟已实现得七七八八。

但是，有些问题直到最后也没能很好地解决。比如，状态机写法的代码放到环形 WS2812B 灯上就是无法点亮灯带。环形灯带本应该和长条型灯带相同，然而即使是相同的代码也无法驱动环形灯显示。开始我以为是硬件问题，在 DDL 前一周又买了另一家的环形灯，赶在 DDL 前实验依然无法点亮。最后只好放弃掌心环形灯。再比如，红外线感应模块被意外烧坏，我们不得不放弃车座红外感应功能。还有拍照显示功能，摄像头相关的代码就是综合不出来，实在是让人气馁。最后我们不得不割舍掉这些功能。

由于大家拖 DDL 的习（恶）惯（习），时间算是比较紧张的，所幸琪葳和智伟在最后时段前完成了模块的突破性进展。最后我们可以算是极限通关

了，大家一起熬了一整个通宵。做出成品的那一刻，我和伙伴们望着紫金港升起的太阳，方悟得人到山前总有路，难事只怕有心人。

（2）宋智伟：

整门课学下来感觉数字系统是门动手能力极强的课程，而在做大作业的过程中发现这门课不仅需要专业知识和动手能力，还需要彼此间的合作，开始的时候由于我们小组是按模块分工做大作业，所以我只把目光注意到如何实现自己模块的功能，带来的结果是我的的确确将自己的部分完成了，但是当我的部分和凯文的拼接时出现了问题，我的输出没有达到凯文输入的要求，之后成为了一个重难点，接下来就是不断地询问助教，得到了解决问题的理论方法，但是在最后实践的时候，出现了问题，输出并不能 1010 的切换，我们只好将凯文的模块的输出作为电源输入播报模块，解决了问题。
这门大作业的完成我还学会了如何烧录代码，如何使用 USB 转 TTL，在不会的时候我感觉自己做不下去了，很担心耽误到同组成员的工作进展，为此熬了好几天的夜，终于在不断地找视频学习到过程中悟了，自己终于把代码完完整整地烧录到了模块上，但是语音识别需要不断地尝试，不断地修改词条，在调试的过程中不知道为什么模块坏了，我当时有点慌，只好及时买了一个，又按照之前的学习，这过程中十分担心烧录不成功，还好在一遍又一遍的尝试之后我终于掌握了这个过程，之后就是视频的录制，在这天晚上熬的很晚，大家都说累但也没有谁说要先走，感觉小组间的合作还是很不错的，还在凌晨将视频剪辑出来，总的来说这次大作业不仅是对学业有了一个新的掌握，还收获了友谊和合作的经验。

（3）王琪葳：

本次数字系统的大作业是目前最“硬核”的一个作业，全程参与下来，感触多多，收获满满，是一次非常难忘的经历。
从无到有，是一个漫长而艰难的过程。一开始的开题内容的讨论，便遇到很大的问题。想法思路虽多，但过于泛滥，难以在现实中具体实现，小组内部也有了想法上的分歧。但是，还是感谢小组成员的积极参与，在深入交流与

探讨的基础上，有所突破，基本确定大致的重点与思路，并在大家集思广益下不断优化，为最终成果的实现打下了基础。

我所分配到的是关于时间、显示方面的工作任务。由于是初次接触，在很多方面上有着很大的误解。原本以为很简单，既然是开源的，那网络资源中的代码下载下来就好，于是花费了很多无用的时间在找到各种各样的代码上面。自信满满的去测试，结果是无一例外的失败，带来的失败感打击还是蛮大的，焦躁、不安的情绪困扰了很久。工作上可以说是毫无进展，尤其是当我入手的第一部分就是 OV7670 摄像头显示，如今想来，那样的做法失败也是不可避免。时间的推进，督促着我必须从中有所反思与思索，我开始静下心来，幸运的找到了相关的学习资料，一个小小的 SCCB 讲解视频就长达 1 小时，我试着去学习，虽然有所收获，但过程说为煎熬不足为过。学习了 IP 核 的设定，学习了相关时钟的对应设置、相关 RAM 的配置，排除掉一个个刺眼的红色 erro 与橙色 critica warning，带着信心再次去实验综合时，失败又给予我沉重打击——不报错，但就是综合不了。询问过两位助教之后，可能是软件的问题，还是没能找到有效的解决办法，也这就意味着，之前的时间投的回报为零。

煎熬之后，便是新的开始，换个功能，从头弄起。前面的经历也不是一无所获，我逐渐学会了面对问题，解决问题。LCD1602 模块一开始其实也并不成功，虽然学习理解后觉得简单，但是当硬件实现与软件代码相结合，总是发生各种问题。代码学习、写好后，正常生成比特流，放入模块中，屏幕不亮——网上搜索解决方案，许多此尝试后，调试背光板亮度，成功。无字符显示，电压不够——先是着手调节电压 3.3v 至 5v 后不亮，认为模块焊接出现问题，重新购买并调试，成功。显示错误——多次实验确认硬件无误的情况下，重新回归代码，观摩网上的基础代码学习，重新自己编写相关细节，成功……一次次的问题的解决的背后是一次次的晚上的爆肝，逐渐学会平静的接受，努力的不断尝试，发现一切问题便迎刃而解。

在之后的模块组装中，我参与到小组其他成员的模块中去，帮助搭建，查看修改代码，充当手模，交流经验的同时也学到了很多不一样的模块知识。有了之间的失败经验的积累，2 天内小组成员同心协力，全力以赴，最终有了完美的结果，喜悦与激动自然不用多说。

实验结束，感觉自己最大的收获不是知识上的，而是处理问题的能力。时刻对自己充满信心，平静的接受，积极的挑战，这种态度这便是我这门课程最大的收获。

六、成员分工及贡献比

梁凯文： 35%

无线电发射接收，灯带效果显示，舵机控制，模块间升降压和电平转换，时钟模块，高压发生器，挥手感应，模块装配协调，实验报告编写，开题答辩主讲。

宋智伟: 31%

语音识别，语音播报，实验报告编写。

王琪葳: 34%

lcd 显示，时钟模块，摄像模块半完成，计时器，最佳手模，实验报告编写，开题答辩 ppt 制作。